

Guía Completa de Despliegue en AWS con Terraform y GitHub Actions (lauracanceleatiempo)

Esta guía documenta paso a paso cómo subir el proyecto a AWS utilizando **Terraform** como Infraestructura como Código (IaC) y **GitHub Actions** para el CI/CD continuo.

1. Servicios y Arquitectura a Utilizar

El proyecto despliega una arquitectura basada en microservicios o contenedores robusta y moderna en AWS empleando los siguientes servicios:

-  **Amazon VPC (Virtual Private Cloud):** Red privada virtual con subredes públicas (para el Balanceador) y privadas (para ECS y RDS).
 -  **Amazon RDS (Relational Database Service):** Instancia de base de datos PostgreSQL gestionada, aislada en subredes privadas.
 -  **Amazon ECR (Elastic Container Registry):** Repositorio privado para almacenar las imágenes Docker (como el `laura-api`).
 -  **Amazon ECS (Elastic Container Service) con Fargate:** Orquestador Serverless donde corren los contenedores, sin necesidad de administrar servidores EC2 subyacentes.
 -  **Amazon ALB (Application Load Balancer):** Balanceador de carga para distribuir el tráfico de internet HTTP/HTTPS hacia las tareas de ECS.
 -  **AWS SSM Parameter Store:** Almacén seguro para guardar la contraseña de la BD y la URI de Keycloak sin exponerlas en el código.
 -  **Amazon CloudWatch Logs:** Servicio de monitoreo para ver los logs generados por la aplicación Spring Boot.
-

2. Estructura de Archivos de Terraform

El código de infraestructura debe estar organizado jerárquicamente en la carpeta `/terraform/`.

Archivos Principales (Raíz de Terraform)

- `main.tf` : Orquestador principal. Aquí se llama a todos los módulos hijos (`network`, `rds`, `ecr`, `ssm`, `ecs`). También declara el proveedor (`aws`) y (opcionalmente) la configuración del backend remoto (`s3`).
- `variables.tf` : Define todas las variables de entrada que los módulos necesitan (ej: `aws_region`, `project_name`, `db_password`).
- `outputs.tf` : Muestra resultados tras el despliegue (como el endpoint del ALB o la URL de la base de datos).
- `terraform.tfvars` : Archivo de valores reales (No se sube a Git). Aquí colocas tus configuraciones como la región, la contraseña secreta, usuario de la base de datos, etc.

Directorio `/modules/`

Contiene la configuración segmentada de la infraestructura:

- `/modules/network/main.tf` : Crea la VPC, Subredes Públicas (2) y Privadas (2), Tablas de rutas, Internet Gateway y NAT Gateway (para que las ECS privadas tengan internet de salida).
- `/modules/rds/main.tf` : Aprovisiona la instancia de PostgreSQL y sus Grupos de Seguridad (solo permite acceso desde dentro de la VPC).

- `/modules/ecr/main.tf` : Crea el repositorio Docker.
 - `/modules/ssm/main.tf` : Sube variables de entorno clave (`SPRING_DATASOURCE_URL` , credenciales) que ECS lee durante el inicio del contenedor.
 - `/modules/ecs/main.tf` : Módulo más complejo. Crea el clúster Fargate, el Security Group del ALB, el Load Balancer, la "Task Definition" (los specs del contenedor como memoria, puerto y variables inyectadas de SSM) y el Servicio ECS.
-

3. Configuración en GitHub Actions (Pipeline CI/CD)

El archivo en `.github/workflows/aws-ecs.yml` automatiza el proceso. Funciona en dos etapas principales o "Jobs":

Job 1: Aprovisionamiento (Terraform)

Se encarga de levantar o actualizar la red, la BD y los clústers. Le pasa los secretos al comando de plan y apply.

- **Secreciones Necesarias en GitHub:**
 - `AWS_ACCESS_KEY_ID` y `AWS_SECRET_ACCESS_KEY` : Cuenta de servicio IAM (preferiblemente con política `AdministratorAccess` para la primera vez).
 - `DB_PASSWORD` : Clave para aprovisionar PostgreSQL.
 - `KEYCLOAK_ISSUER_URI` : URL para validar los tokens.

Job 2: Empaquetar y Desplegar (Docker & ECS)

Inicia compilando la app con Maven, armando la imagen de Docker, y subiéndola a Amazon ECR. Luego notifica al cluster ECS para que descargue la nueva imagen y la reinicie sin tiempo de inactividad temporal (Zero Downtime Deployment).

- **Parámetros Ambientales Importantes:**
 - `ECR_REPOSITORY` : Debe coincidir con el creado por el módulo de Terraform.
 - `ECS_CLUSTER` y `ECS_SERVICE` : Identificadores únicos del clúster de destino.
 - `ECS_TASK_DEFINITION` : El nombre unívoco de la Tarea Fargate asignada.
-

4. Paso a Paso para Desplegar

 **Advertencia:** Asegúrate de que tienes instalado Docker y Terraform en tu sistema si decides probar en local antes de subir los cambios al Pipeline.

Fase 1: Creación del Usuario y Preparación

1. Ve a la consola **AWS IAM > Usuarios** y crea un nuevo usuario.
2. Asigne temporalmente la política preexistente `AdministratorAccess` .
3. Genérale **Credenciales de Seguridad (Access Keys)**. Guarda las claves.
4. En **GitHub > Settings de tu repositorio > Secrets and variables > Actions**, crea los Secretos:
 - `AWS_ACCESS_KEY_ID` : Tu access key de IAM.
 - `AWS_SECRET_ACCESS_KEY` : Tu secret key de IAM.
 - `DB_PASSWORD` : Una contraseña fuerte para la BD de producción.
 - `KEYCLOAK_ISSUER_URI` : El link a donde apuntará Keycloak.

Fase 2: Aplicación Manual (Opcional pero Recomendado)

Antes de presionar "push" para que GitHub corra todo en frío, es una excelente práctica levantar la base de infraestructura desde tu computadora local.

1. Abre tu terminal y ve a `cd terraform/`.
2. Renombra el archivo `terraform.tfvars.example` a `terraform.tfvars` y completa los datos dentro (contraseñas).
3. Ejecuta `terraform init` (descarga plugins).
4. Ejecuta `terraform plan` (visualizas qué va a crear AWS).
5. Ejecuta `terraform apply -auto-approve` (espera unos ~10 a 15 minutos en lo que se crea la RDS y el NAT Gateway).

Fase 3: Despliegue de la Aplicación en la Nube

Una vez la ruta de Terraform existe:

1. Haz un `git add .`, y un commit a la rama `main`.
2. Al pushear (`git push origin main`), GitHub Actions entra en acción automáticamente.
3. Puedes ver en vivo desde la pestaña "Actions" en GitHub cómo:
 - Valida el Terraform (`Job 1`).
 - Compila el código, crea la imagen Docker, baja el `task-definition.json` de ECS, lo actualiza con una nueva imagen, y lanza la aplicación.
4. ¡Terminaste! Si copias la URL (DNS Público) provisto por el ALB que arrojó el output, tu Backend estará accesible globalmente en Internet.

5. Mantenimiento y Cuidados Críticos

- **Costos de NAT Gateway y RDS:** Al usar arquitecturas basadas en *Subredes Privadas*, Terraform levanta un componente llamado NAT Gateway, que conlleva un costo por hora independientemente de su uso al igual que RDS. No dejes proyectos de laboratorio encendidos de por vida con `apply`.
- **Destrucción Total (Limpieza):** Si el proyecto es para la universidad/prueba y ya la validaron, **DEBES** rodar `terraform destroy -auto-approve` para eliminar los recursos de AWS y detener todos los cobros inmediatamente.
- **Estado de Terraform (`terraform.tfstate`):** Este archivo JSON controla y conoce qué IDs ha creado. Si lo pierdes, Terraform no sabrá qué modificar a futuro. Si vas a trabajar en equipo, considera usar "Backend S3" en lugar de archivos locales.